UniCesumar
Engenharia de Software

Manutenção de Software

Código Limpo – Funções

- O que faz o trecho de código a seguir?

Código Limpo – Funções

```
public static String testableHtml(
  PageData pageData,
  boolean includeSuiteSetup
) throws Exception {
  WikiPage wikiPage = pageData.getWikiPage();
  StringBuffer buffer = new StringBuffer();
  if (pageData.hasAttribute("Test")) {
    if (includeSuiteSetup) {
      WikiPage suiteSetup =
       PageCrawlerImpl.getInheritedPage(
               SuiteResponder.SUITE_SETUP_NAME, wikiPage
       );
      if (suiteSetup != null) {
       WikiPagePath pagePath =
         suiteSetup.getPageCrawler().getFullPath(suiteSetup);
       String pagePathName = PathParser.render(pagePath);
       buffer.append("!include -setup .")
             .append(pagePathName)
             .append("\n");
      }
    }
    WikiPage setup =
      PageCrawlerImpl.getInheritedPage("SetUp", wikiPage);
    if (setup != null) {
      WikiPagePath setupPath =
        wikiPage.getPageCrawler().getFullPath(setup);
      String setupPathName = PathParser.render(setupPath);
      buffer.append("!include -setup .")
            .append(setupPathName)
            .append("\n");
    }
  }
```

```
  buffer.append(pageData.getContent());
  if (pageData.hasAttribute("Test")) {
    WikiPage teardown =
      PageCrawlerImpl.getInheritedPage("TearDown", wikiPage);
    if (teardown != null) {
      WikiPagePath tearDownPath =
        wikiPage.getPageCrawler().getFullPath(teardown);
      String tearDownPathName = PathParser.render(tearDownPath);
      buffer.append("\n")
            .append("!include -teardown .")
            .append(tearDownPathName)
            .append("\n");
    }
     if (includeSuiteSetup) {
       WikiPage suiteTeardown =
         PageCrawlerImpl.getInheritedPage(
                 SuiteResponder.SUITE_TEARDOWN_NAME,
                 wikiPage
         );
       if (suiteTeardown != null) {
         WikiPagePath pagePath =
           suiteTeardown.getPageCrawler().getFullPath (suiteTeardown);
         String pagePathName = PathParser.render(pagePath);
         buffer.append("!include -teardown .")
               .append(pagePathName)
               .append("\n");
       }
     }
  }
  pageData.setContent(buffer.toString());
  return pageData.getHtml();
}
```

Funções

- Muito conteúdo (append, crawler, parser, buffer, etc)

- Código duplicado

- Condicionais aninhadas

Funções

- Refatorando
- E agora?

```
public static String renderPageWithSetupsAndTeardowns(
  PageData pageData, boolean isSuite
) throws Exception {
  boolean isTestPage = pageData.hasAttribute("Test");
  if (isTestPage) {
    WikiPage testPage = pageData.getWikiPage();
    StringBuffer newPageContent = new StringBuffer();
    includeSetupPages(testPage, newPageContent, isSuite);
    newPageContent.append(pageData.getContent());
    includeTeardownPages(testPage, newPageContent, isSuite);
    pageData.setContent(newPageContent.toString());
  }

  return pageData.getHtml();
}
```

Funções

- Setups e teardown em página de testes

- Renderiza HTML

- Teste web

- Extrair informação da segunda função é mais fácil que da primeira

Funções

- O que torna uma função fácil de ler?

Funções

- Como podemos fazer uma função comunicar sua intenção?

Funções

- Quais atributos uma função deve ter?

Funções devem ser pequenas

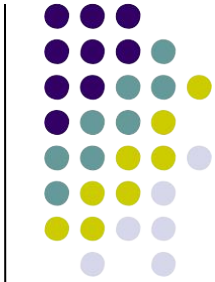
- Primeira regra

- Idealmente: if, elses, whiles devem ter uma linha

```
public static String renderPageWithSetupsAndTeardowns(
  PageData pageData, boolean isSuite
) throws Exception {
  boolean isTestPage = pageData.hasAttribute("Test");
  if (isTestPage) {
    WikiPage testPage = pageData.getWikiPage();
    StringBuffer newPageContent = new StringBuffer();
    includeSetupPages(testPage, newPageContent, isSuite);
    newPageContent.append(pageData.getContent());
    includeTeardownPages(testPage, newPageContent, isSuite);
    pageData.setContent(newPageContent.toString());
  }

  return pageData.getHtml();
}
```

Funções devem ser pequenas



```java
public static String renderPageWithSetupsAndTeardowns(
  PageData pageData, boolean isSuite
) throws Exception {
  boolean isTestPage = pageData.hasAttribute("Test");
  if (isTestPage) {
    WikiPage testPage = pageData.getWikiPage();
    StringBuffer newPageContent = new StringBuffer();
    includeSetupPages(testPage, newPageContent, isSuite);
    newPageContent.append(pageData.getContent());
    includeTeardownPages(testPage, newPageContent, isSuite);
    pageData.setContent(newPageContent.toString());
  }

  return pageData.getHtml();
}
```

```java
public static String renderPageWithSetupsAndTeardowns(
  PageData pageData, boolean isSuite
) throws Exception {
  boolean isTestPage = pageData.hasAttribute("Test");
  if (isTestPage(pageData))
    includeSetupAndTeardownPages(pageData, isSuite);
  return pageData.getHtml();
}
```

Funções devem realizar apenas uma tarefa

- A primeira função realiza várias tarefas

- Cria buffers
- Busca páginas
- Busca heranças
- Faz parser
- Acrescenta string
- Gera HTML

Funções devem realizar apenas uma tarefa

- Possui vários níveis de abstração

- Alto nível: getHtml()

- Nível intermediário: PathParser.render(PagePath)

- Baixo nível: append(“\n”)

Funções devem realizar apenas uma tarefa

- A última função faz apenas uma coisa

- Adiciona setups e teardowns em páginas de testes

```java
public static String testableHtml(
  PageData pageData,
  boolean includeSuiteSetup
) throws Exception {
  WikiPage wikiPage = pageData.getWikiPage();
  StringBuffer buffer = new StringBuffer();
  if (pageData.hasAttribute("Test")) {
    if (includeSuiteSetup) {
      WikiPage suiteSetup =
       PageCrawlerImpl.getInheritedPage(
               SuiteResponder.SUITE_SETUP_NAME, wikiPage
       );
      if (suiteSetup != null) {
       WikiPagePath pagePath =
         suiteSetup.getPageCrawler().getFullPath(suiteSetup);
       String pagePathName = PathParser.render(pagePath);
       buffer.append("!include -setup .")
             .append(pagePathName)
             .append("\n");
      }
    }
    WikiPage setup =
      PageCrawlerImpl.getInheritedPage("SetUp", wikiPage);
    if (setup != null) {
      WikiPagePath setupPath =
        wikiPage.getPageCrawler().getFullPath(setup);
      String setupPathName = PathParser.render(setupPath);
      buffer.append("!include -setup .")
            .append(setupPathName)
            .append("\n");
    }
  }
```

```java
  buffer.append(pageData.getContent());
  if (pageData.hasAttribute("Test")) {
    WikiPage teardown =
      PageCrawlerImpl.getInheritedPage("TearDown", wikiPage);
    if (teardown != null) {
      WikiPagePath tearDownPath =
        wikiPage.getPageCrawler().getFullPath(teardown);
      String tearDownPathName = PathParser.render(tearDownPath);
      buffer.append("\n")
            .append("!include -teardown .")
            .append(tearDownPathName)
            .append("\n");
    }
     if (includeSuiteSetup) {
       WikiPage suiteTeardown =
         PageCrawlerImpl.getInheritedPage(
                 SuiteResponder.SUITE_TEARDOWN_NAME,
                 wikiPage
         );
       if (suiteTeardown != null) {
         WikiPagePath pagePath =
           suiteTeardown.getPageCrawler().getFullPath (suiteTeardown);
         String pagePathName = PathParser.render(pagePath);
         buffer.append("!include -teardown .")
               .append(pagePathName)
               .append("\n");
       }
     }
  }
  pageData.setContent(buffer.toString());
  return pageData.getHtml();
}
```

```java
public static String renderPageWithSetupsAndTeardowns(
  PageData pageData, boolean isSuite
) throws Exception {
  boolean isTestPage = pageData.hasAttribute("Test");
  if (isTestPage(pageData))
    includeSetupAndTeardownPages(pageData, isSuite);
  return pageData.getHtml();
}
```

Nomes em funções

- Nomes descritivos são melhores que nomes curtos e enigmáticos

- Definir um nome descritivo faz com que um comentário seja desnecessário

Nomes em funções

- Nomear entidades não é uma tarefa fácil (apesar de parecer fácil)
  - IDEs facilitam a troca de nomes

- Seja consistente nos nomes
  - Utilizar mesmo prefixo/sufixo

Argumentos em funções

- O ideal é não utilizar

- Se necessário: um ou dois

- Evitar três ou mais

Argumentos em funções

- Por que evitar?

- Torna a função mais complexa

- Dificulta a criação de testes (todas as combinações devem ser testadas)

Um argumento

- Realizar uma pergunta sobre o argumento

- boolean fileExists(“MyFile.txt”)

Um argumento

- Operar no argumento, transformar e retornar ele

- inputStream fileOpen(“MyFile.txt”)

Um argumento

- Alterar o estado do sistema

- void passwordAttemptFailedNtimes(int attempts)

Um argumento

- void transform(StringBuffer out)

- ou

- StringBuffer transform(StringBuffer in)

Um argumento

- void transform(StringBuffer out)

- Pode confundir entrada com saída

- StringBuffer transform(StringBuffer in)

- Retorna o valor correto, dado a entrada

Efeitos colaterais

- Evitar funções que fazem tarefas “escondidas”

- Exemplos:

- Mudanças inesperadas em variáveis, parâmetros ou atributos

- Realização de tarefas distintas do seu objetivo

Efeitos colaterais

- Qual o efeito colateral da seguinte função?

```
public class UserValidator {
  private Cryptographer cryptographer;

  public boolean checkPassword(String userName, String password) {
    User user = UserGateway.findByName(userName);
    if (user != User.NULL) {
      String codedPhrase = user.
      getPhraseEncodedByPassword();
      String phrase = cryptographer.decrypt(codedPhrase, password);
      if ("Valid Password".equals(phrase)) {
        Session.initialize();
        return true;
      }
    }
    return false;
  }
}
```

Efeitos colaterais

- Qual o efeito colateral da seguinte função?

```
public class UserValidator {
  private Cryptographer cryptographer;

  public boolean checkPassword(String userName, String password) {
    User user = UserGateway.findByName(userName);
    if (user != User.NULL) {
      String codedPhrase = user.
      getPhraseEncodedByPassword();
      String phrase = cryptographer.decrypt(codedPhrase, password);
      if ("Valid Password".equals(phrase)) {
        Session.initialize();
        return true;
      }
    }
    return false;
  }
}
```

Efeitos colaterais

- De acordo com o nome, a função deveria validar o password, no entanto também inicializa a sessão

```
public class UserValidator {
  private Cryptographer cryptographer;

  public boolean checkPassword(String userName, String password) {
    User user = UserGateway.findByName(userName);
    if (user != User.NULL) {
      String codedPhrase = user.
      getPhraseEncodedByPassword();
      String phrase = cryptographer.decrypt(codedPhrase, password);
      if ("Valid Password".equals(phrase)) {
        Session.initialize();
        return true;
      }
    }
    return false;
  }
}
```

- Renomear: **checkPasswordAndInitializeSession()**

Funções e repetições

- Não repita a si mesmo

- Duplicação só traz problemas

- Vários princípios e práticas foram criados para evitar ou eliminar duplicação de código

- Funções, métodos, OO, programação estruturada, programação orientada a aspectos, em parte são, estratégias para eliminar duplicação

Onde está a duplicação?

```
public static String testableHtml(
  PageData pageData,
  boolean includeSuiteSetup
) throws Exception {
  WikiPage wikiPage = pageData.getWikiPage();
  StringBuffer buffer = new StringBuffer();
  if (pageData.hasAttribute("Test")) {
    if (includeSuiteSetup) {
      WikiPage suiteSetup =
       PageCrawlerImpl.getInheritedPage(
               SuiteResponder.SUITE_SETUP_NAME, wikiPage
       );
      if (suiteSetup != null) {
       WikiPagePath pagePath =
         suiteSetup.getPageCrawler().getFullPath(suiteSetup);
       String pagePathName = PathParser.render(pagePath);
       buffer.append("!include -setup .")
             .append(pagePathName)
             .append("\n");
      }
    }
    WikiPage setup =
      PageCrawlerImpl.getInheritedPage("SetUp", wikiPage);
    if (setup != null) {
      WikiPagePath setupPath =
        wikiPage.getPageCrawler().getFullPath(setup);
      String setupPathName = PathParser.render(setupPath);
      buffer.append("!include -setup .")
            .append(setupPathName)
            .append("\n");
    }
  }
```

```
  buffer.append(pageData.getContent());
  if (pageData.hasAttribute("Test")) {
    WikiPage teardown =
      PageCrawlerImpl.getInheritedPage("TearDown", wikiPage);
    if (teardown != null) {
      WikiPagePath tearDownPath =
        wikiPage.getPageCrawler().getFullPath(teardown);
      String tearDownPathName = PathParser.render(tearDownPath);
      buffer.append("\n")
            .append("!include -teardown .")
            .append(tearDownPathName)
            .append("\n");
    }
     if (includeSuiteSetup) {
       WikiPage suiteTeardown =
         PageCrawlerImpl.getInheritedPage(
                 SuiteResponder.SUITE_TEARDOWN_NAME,
                 wikiPage
         );
       if (suiteTeardown != null) {
         WikiPagePath pagePath =
           suiteTeardown.getPageCrawler().getFullPath (suiteTeardown);
         String pagePathName = PathParser.render(pagePath);
         buffer.append("!include -teardown .")
               .append(pagePathName)
               .append("\n");
       }
     }
   }
   pageData.setContent(buffer.toString());
   return pageData.getHtml();
}
```

Onde está a duplicação?

- 4 locais para mudar – MANUTENÇÃO

- Não é fácil de ver

- 4 oportunidades de introduzir erros

Conclusões

- Funções devem ser refinadas ao longo do tempo

- Primeira versão nunca é perfeita
  - Geralmente, longa e complicada
  - Talvez com loops aninhados e longas listas de argumentos
  - Possivelmente com nomes ruins e código duplicado

Conclusões

- Escrever testes

- Faz com que as funções possam ser refinadas, divididas, renomeadas, removidas as duplicações, com a garantia dos testes